

TASK 1: ELIMINATING REPETITIVE LOGIC

ORIGINAL CODE HAD REPEATED AREA AND PERIMETER CALCULATIONS. USED AI SUGGESTION TO CREATE FUNCTIONS FOR REUSE.

REFACTORED CODE (TASK1_REFACTORED.PY):

```
def calculate_area(length,
width): """CALCULATE AREA OF
RECTANGLE.""" return length *
width

def calculate_perimeter(length,
width): """CALCULATE PERIMETER OF
RECTANGLE.""" return 2 * (length +
width)

# FIRST RECTANGLE LENGTH = 8 WIDTH = 4
print("AREA:", calculate_area(length,
width)) print("PERIMETER:",
calculate_perimeter(length, width))

# SECOND RECTANGLE LENGTH = 12
width = 6 print("AREA:",
calculate_area(length, width))
print("PERIMETER:", calculate_perimeter(length, width))
```

OUTPUT:

```
$ python task1_refactored.py
AREA: 32
PERIMETER: 24
AREA: 72
PERIMETER: 36
```

TASK 2: MODULARIZING INLINE CALCULATIONS

REPEATED DISCOUNT LOGIC EXTRACTED INTO FUNCTION PER AI REFACTOR SUGGESTION.

REFACTORED CODE (TASK2_REFACTORED.PY):

```
def apply_discount(amount, rate=0.10):  
    """APPLY DISCOUNT TO AMOUNT AND RETURN FINAL  
    AMOUNT."""  
    discount = amount * rate  
    return amount - discount  
  
# FIRST AMOUNT  
amount = 1000  
print("FINAL AMOUNT:",  
      apply_discount(amount))  
  
# SECOND AMOUNT  
amount = 2000  
print("FINAL AMOUNT:", apply_discount(amount))
```

OUTPUT:

```
$ python task2_refactored.py  
FINAL AMOUNT: 900.0  
FINAL AMOUNT: 1800.0
```

TASK 3: IMPROVING LOOP AND CONDITIONAL PERFORMANCE

NESTED LOOPS INEFFICIENT. AI SUGGESTED SET FOR O(1) LOOKUPS.

ORIGINAL EXECUTION TEST:

```
$ time python task3_original.py  
CHARAN FOUND  
ESHA NOT  
FOUND BHAVYA  
FOUND REAL  
0m0.000s
```

REFACTORED CODE (TASK3_REFACTORED.PY):

```
STUDENTS = ["ANIL", "BHAVYA", "CHARAN", "DIVYA"]
CHECK_LIST = ["CHARAN", "ESHA", "BHAVYA"]
```

```
# CONVERT TO SET FOR FAST LOOKUP
STUDENTS_SET = SET(STUDENTS)
```

```
FOR NAME IN
CHECK_LIST:  IF
NAME IN
STUDENTS_SET:
PRINT(NAME, "FOUND")
ELSE:
    PRINT(NAME, "NOT FOUND")
```

TERMINAL OUTPUT AND TEST:

```
$ PYTHON TASK3_REFACTORED.PY
CHARAN FOUND
ESHA NOT FOUND
BHAVYA FOUND
$ TIME PYTHON TASK3_REFACTORED.PY REAL 0M0.000S
```

FASTER LOOKUPS, SAME OUTPUT. SET IMPROVES PERFORMANCE FOR LARGER LISTS.

TASK 4: REPLACING HARDCODED VALUES WITH CONSTANTS

HARDCODED NUMBERS REPLACED WITH CONSTANTS AS AI RECOMMENDED.

REFACTORED CODE (TASK4_REFACTORED.PY):

```
PRINCIPAL1 = 1000
PRINCIPAL2 = 2000
TIME = 2
RATE = 5 # PERCENT

DEF SIMPLE_INTEREST(P, T,
R):  RETURN (P * T * R) / 100
```

```
PRINT("SIMPLE INTEREST:", SIMPLE_INTEREST(PRINCIPAL1, TIME, RATE))  
PRINT("SIMPLE INTEREST:", SIMPLE_INTEREST(PRINCIPAL2, TIME, RATE))
```

TERMINAL OUTPUT:

```
$ PYTHON TASK4_REFACTORED.PY  
SIMPLE INTEREST: 100.0  
SIMPLE INTEREST: 200.0
```

MATCHES ORIGINAL. EASY TO UPDATE RATE OR TIME NOW.

TASK 5: ENHANCING VARIABLE NAMING AND CODE CLARITY

RENAMED VARIABLES AND ADDED COMMENTS BASED ON AI SUGGESTIONS FOR READABILITY.

REFACTORED CODE (TASK5_REFACTORED.PY):

```
FIRST_NUMBER = 50 # STARTING VALUE FOR ADDITION  
SECOND_NUMBER = 30 # VALUE TO ADD TO FIRST NUMBER  
TOTAL_SUM = FIRST_NUMBER + SECOND_NUMBER # SUM OF THE TWO NUMBERS  
  
PRINT(TOTAL_SUM) # OUTPUT THE RESULT
```

OUTPUT:

```
$ PYTHON TASK5_REFACTORED.PY  
80
```

SAME OUTPUT. MUCH CLEARER WHAT EACH VARIABLE MEANS.